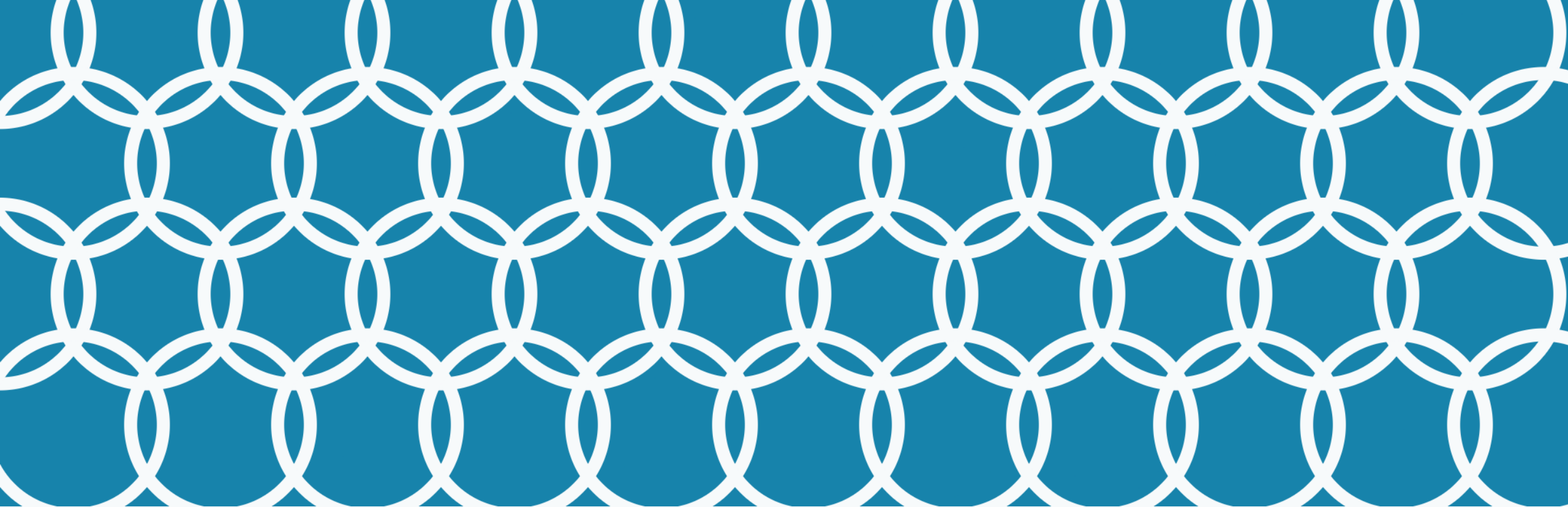




Computer Graphics (4)

Lab 4 • Color Systems: RGB, CMY and HSV

| Agenda

- 01 Color in OpenGL
- 03 Color swatches
- 05 Additive mixing (RGB)
- 07 The HSV model
- 09 HSV to RGB
- 11 Complements and pitfalls
- 02 The RGB cube
- 04 Smooth shading
- 06 Subtractive mixing (CMY)
- 08 RGB to HSV
- 10 Color wheel
- 12 Lab tasks

| Color in OpenGL

```
// background color
glClearColor(0.0, 0.0, 1.0, 1.0);

// current drawing color
glColor3f(1.0, 0.0, 0.0);

// the same red, as bytes
glColor3ub(255, 0, 0);
```

Each component is a **float from 0.0 to 1.0** (or a byte from 0 to 255 with the **ub** version).

Values outside the range are **clamped**:

`glColor3f(255, 0, 0)` is just red, not "255 times red".

The color is a **state**: every vertex after `glColor` uses it until you change it.

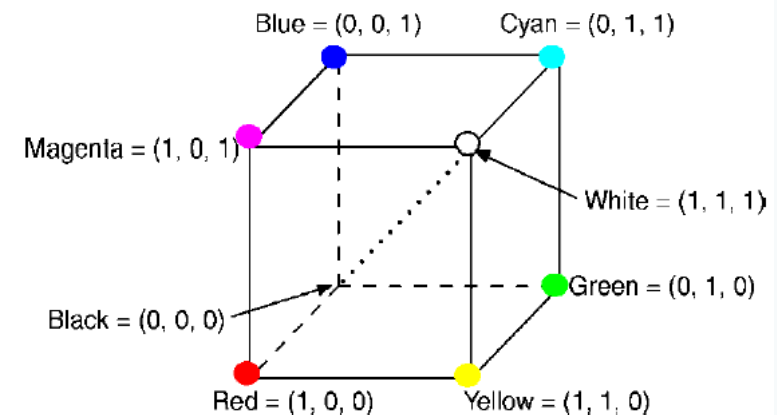
The RGB cube

Corner	R	G	B
Black	0	0	0
Red	1	0	0
Green	0	1	0
Blue	0	0	1
Yellow	1	1	0
Cyan	0	1	1
Magenta	1	0	1
White	1	1	1

Grays live on the diagonal

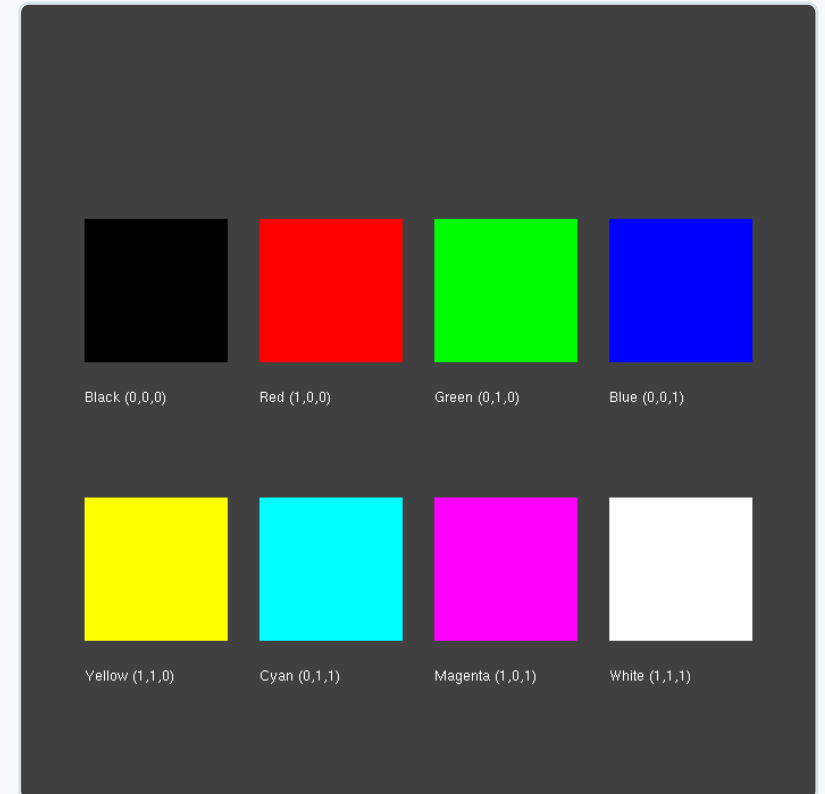
When $R = G = B$ the color has no hue: it is a gray level between black (0, 0, 0) and white (1, 1, 1).

Mixing light is **additive**: adding all three primaries gives white, and no light at all gives black.



Color swatches

```
float colors[8][3] = {  
    {0,0,0}, {1,0,0}, {0,1,0}, {0,0,1},  
    {1,1,0}, {0,1,1}, {1,0,1}, {1,1,1}};  
  
for (int i = 0; i < 8; i++) {  
    float x = 8 + (i % 4) * 22;  
    float y = 55 - (i / 4) * 35;  
    glColor3f(colors[i][0], colors[i][1], colors[i][2]);  
    glBegin(GL_QUADS);  
    glVertex2f(x, y);  
    glVertex2f(x, y + 18);  
    glVertex2f(x + 18, y + 18);  
    glVertex2f(x + 18, y);  
    glEnd();  
}
```

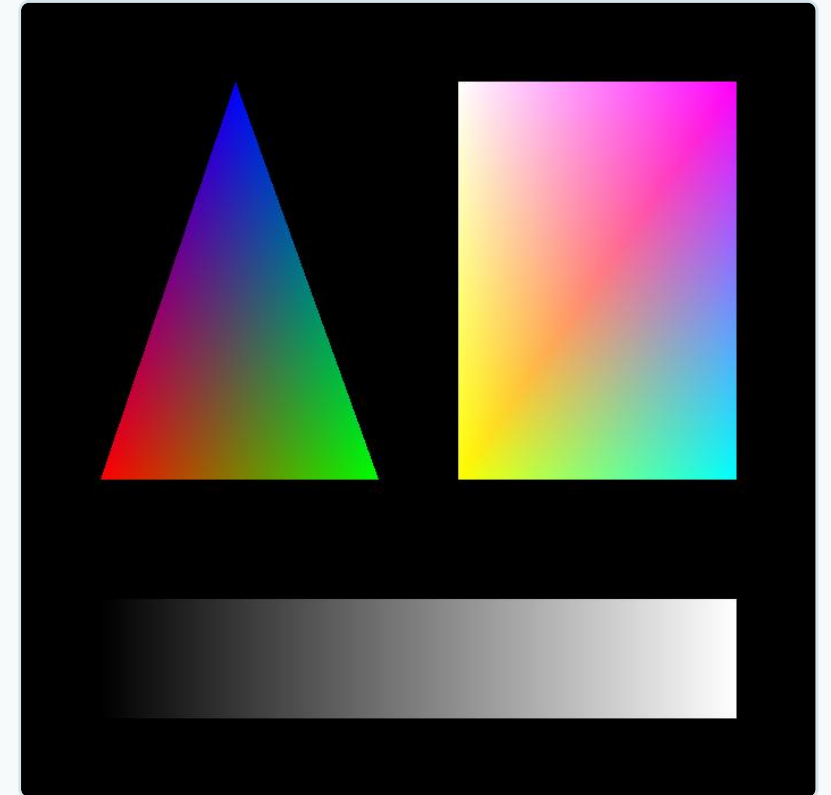


Smooth shading

```
// in init2D(): colors blend between vertices
glShadeModel(GL_SMOOTH);

glBegin(GL_TRIANGLES);
    glColor3f(1, 0, 0); glVertex2i(10, 40);
    glColor3f(0, 1, 0); glVertex2i(45, 40);
    glColor3f(0, 0, 1); glVertex2i(27, 90);
glEnd();

glBegin(GL_QUADS);
    glColor3f(0, 0, 0); glVertex2i(10, 10);
    glVertex2i(10, 25);
    glColor3f(1, 1, 1); glVertex2i(90, 25);
    glVertex2i(90, 10);
glEnd();
```



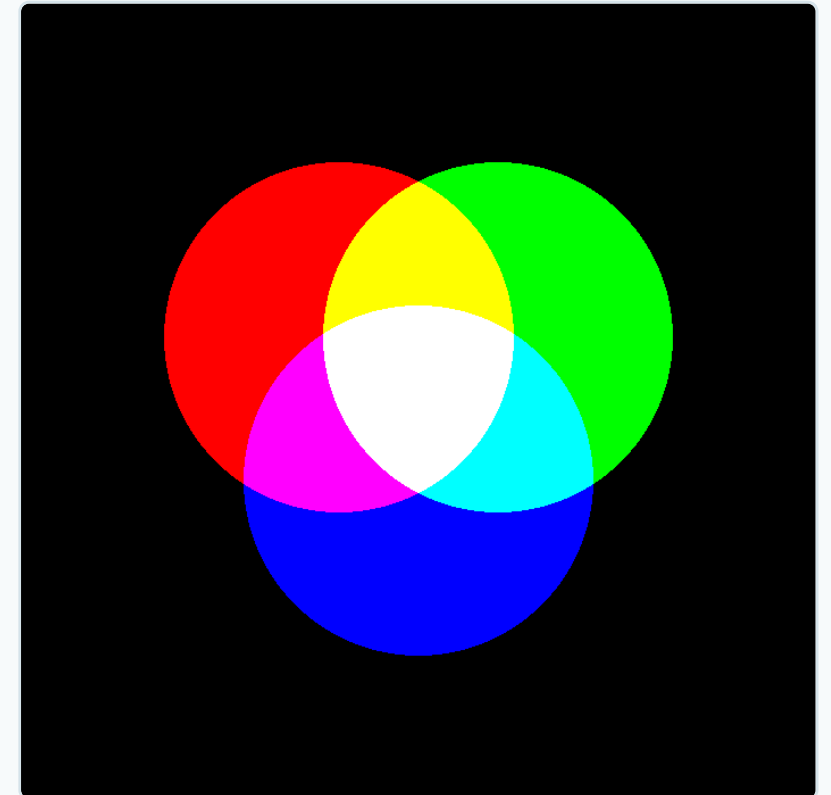
A color set before a vertex belongs to that vertex. OpenGL **interpolates** in between.

Additive mixing (RGB)

```
// init2D(): start from black  
glClearColor(0, 0, 0, 1);  
glEnable(GL_BLEND);  
glBlendFunc(GL_ONE, GL_ONE);  
// result = source + destination
```

```
glColor3f(1, 0, 0); DrawCircle(40, 58, 22, 100);  
glColor3f(0, 1, 0); DrawCircle(60, 58, 22, 100);  
glColor3f(0, 0, 1); DrawCircle(50, 40, 22, 100);
```

$R + G = \text{yellow}$ · $G + B = \text{cyan}$ · $R + B = \text{magenta}$ · $R + G + B = \text{white}$



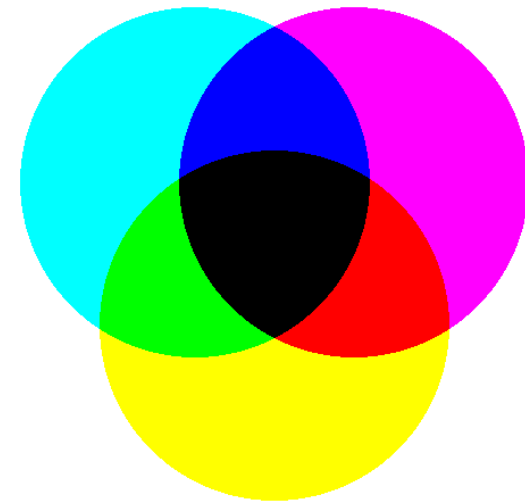
Subtractive mixing (CMY)

```
// OpenGL only knows RGB: R = 1 - C
void setColorCMY(float c, float m, float y) {
    glColor3f(1.0 - c, 1.0 - m, 1.0 - y);
}

// init2D(): ink on WHITE paper
glClearColor(1, 1, 1, 1);
glEnable(GL_BLEND);
glBlendFunc(GL_DST_COLOR, GL_ZERO);
// result = source * destination

setColorCMY(1, 0, 0); DrawCircle(40, 58, 22, 100);
```

$C + M = \text{blue}$ · $M + Y = \text{red}$ · $C + Y = \text{green}$ · $C + M + Y = \text{black}$



CMY = 1 – RGB

Color	RGB	CMY
Red	(1, 0, 0)	(0, 1, 1)
Green	(0, 1, 0)	(1, 0, 1)
Blue	(0, 0, 1)	(1, 1, 0)
White	(1, 1, 1)	(0, 0, 0)
Black	(0, 0, 0)	(1, 1, 1)
Gray	(0.5, 0.5, 0.5)	(0.5, 0.5, 0.5)

Light versus ink

RGB adds light to black: screens, key color white.

CMY(K) removes light from white: printers, key color black.

The RGB gamut is larger than CMYK, so some screen colors lose vibrancy when printed.

| The HSV model

Hue

0° – 360°

The kind of color: red 0°, green 120°, blue 240°.

Saturation

0 – 100%

Colorfulness: 0 is gray, 100 is the purest color.

Value

0 – 100%

Brightness: 0 is black, 100 is the brightest.

Red = (0°, 100%, 100%) · Green = (120°, 100%, 100%) · Blue = (240°, 100%, 100%) · Gray = (0°, 0%, 50%)

RGB to HSV

```
void rgbToHSV(float r, float g, float b,
             float &h, float &s, float &v)
{
    r /= 255.0; g /= 255.0; b /= 255.0;
    float cmax = fmaxf(r, fmaxf(g, b));
    float cmin = fminf(r, fminf(g, b));
    float diff = cmax - cmin;

    if (diff == 0)    h = 0;
    else if (cmax == r) h = fmodf(60 * ((g - b) / diff) + 360, 360);
    else if (cmax == g) h = fmodf(60 * ((b - r) / diff) + 120, 360);
    else             h = fmodf(60 * ((r - g) / diff) + 240, 360);

    s = (cmax == 0) ? 0 : (diff / cmax) * 100;
    v = cmax * 100;
}
```

Step 3:

Hue calculation :

if cmax and cmin equal 0, then h = 0

*if cmax equal r then compute $h = (60 * ((g - b) / \text{diff}) + 360) \% 360$*

*if cmax equal g then compute $h = (60 * ((b - r) / \text{diff}) + 120) \% 360$*

*if cmax equal b then compute $h = (60 * ((r - g) / \text{diff}) + 240) \% 360$*

Step 1 , 2:

$$R' = R/255$$

$$G' = G/255$$

$$B' = B/255$$

$$C_{max} = \max(R', G', B')$$

$$C_{min} = \min(R', G', B')$$

$$\Delta = C_{max} - C_{min}$$

Step 4:

Saturation computation :

if cmax = 0, then s = 0

*if cmax does not equal 0 then compute $s = (\text{diff}/c_{max}) * 100$*

Step 5:

Value computation :

$$v = c_{max} * 100$$

Five steps

1 Divide r, g, b by 255

2 Find cmax, cmin and diff

3 Hue depends on which channel is the maximum

4 Saturation = diff / cmax

5 Value = cmax

h in degrees, s and v in percent, as in the lecture.

Example: (45, 215, 0) to HSV

Step	Result
1 Divide by 255	$R' = 0.17647 \cdot G' = 0.84313 \cdot B' = 0$
2 Max, min, diff	$c_{\max} = G' = 0.84313 \cdot c_{\min} = 0 \cdot \text{diff} = 0.84313$
3 Hue (cmax is G')	$60 \times ((0 - 0.17647) / 0.84313) + 120 = 107.44^\circ$
4 Saturation	$0.84313 / 0.84313 \times 100 = 100$
5 Value	$0.84313 \times 100 = 84.31$

```
rgbToHSV(45, 215, 0, h, s, v);
```

```
// h = 107.44, s = 100.00, v = 84.31
```

HSV to RGB

```
void hsvToRGB(float h, float s, float v,  
             float &r, float &g, float &b)  
{  
    h = fmodf(h, 360);  
    if (h < 0)    h += 360;  
    s /= 100.0; v /= 100.0;  
    float C = v * s;  
    float X = C * (1 - fabsf(fmodf(h / 60.0, 2) - 1));  
    float m = v - C;  
    float r1, g1, b1;  
    if (h < 60)    { r1 = C; g1 = X; b1 = 0; }  
    else if (h < 120) { r1 = X; g1 = C; b1 = 0; }  
    else if (h < 180) { r1 = 0; g1 = C; b1 = X; }  
    else if (h < 240) { r1 = 0; g1 = X; b1 = C; }  
    else if (h < 300) { r1 = X; g1 = 0; b1 = C; }  
    else          { r1 = C; g1 = 0; b1 = X; }  
    r = (r1 + m) * 255; g = (g1 + m) * 255; b = (b1 + m) * 255;  
}
```

What the letters mean

C the chroma: how far the color is from gray

X the second largest component

m the amount added to every channel to match the value

Each 60° sector of the hue circle puts C, X and 0 into a different channel.

HSV TO RGB

- When $0 \leq H < 360$, $0 \leq S \leq 1$ and $0 \leq V \leq 1$:
- $C = V \times S$
- $X = C \times (1 - |(H / 60^\circ) \% 2 - 1|)$
- $m = V - C$

$$(R', G', B') = \begin{cases} (C, X, 0) & , 0^\circ \leq H < 60^\circ \\ (X, C, 0) & , 60^\circ \leq H < 120^\circ \\ (0, C, X) & , 120^\circ \leq H < 180^\circ \\ (0, X, C) & , 180^\circ \leq H < 240^\circ \\ (X, 0, C) & , 240^\circ \leq H < 300^\circ \\ (C, 0, X) & , 300^\circ \leq H < 360^\circ \end{cases}$$

$$(R, G, B) = ((R' + m) \times 255, (G' + m) \times 255, (B' + m) \times 255)$$

| Example: (280°, 50%, 90%) to RGB

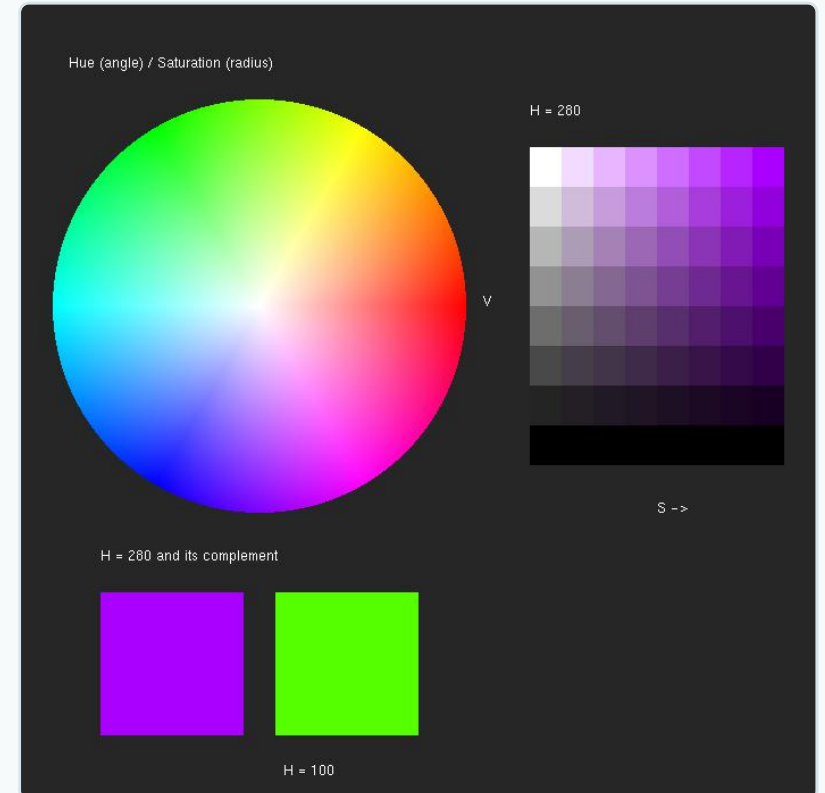
Step	Result
1 Normalize	$S = 0.5 \cdot V = 0.9$
2 Chroma	$C = V \times S = 0.45$
3 Second component	$X = 0.45 \times (1 - (280 / 60) \% 2 - 1) = 0.30$
4 Offset	$m = V - C = 0.45$
5 Sector 240° to 300°	$(R', G', B') = (X, 0, C) = (0.30, 0, 0.45)$
6 Add m, scale by 255	$(R, G, B) = (191, 115, 230)$

```
hsvToRGB(280, 50, 90, r, g, b);  
// r = 191, g = 115, b = 230
```

Color wheel

```
void DrawColorWheel(float cx, float cy, float r) {  
    glBegin(GL_TRIANGLES);  
    for (int i = 0; i < 360; i++) {  
        float a0 = 2 * 3.1415926f * i / 360;  
        float a1 = 2 * 3.1415926f * (i + 1) / 360;  
        setColorHSV(i, 0, 100); // center  
        glVertex2f(cx, cy);  
        setColorHSV(i, 100, 100); // edge 1  
        glVertex2f(cx + r * cosf(a0), cy + r * sinf(a0));  
        setColorHSV(i + 1, 100, 100); // edge 2  
        glVertex2f(cx + r * cosf(a1), cy + r * sinf(a1));  
    }  
    glEnd();  
}
```

360 thin triangles: **hue** is the angle, **saturation** grows from the center to the edge.



| Complementary colors

Hue	Complement (H + 180°)
Red 0°	Cyan 180°
Yellow 60°	Blue 240°
Green 120°	Magenta 300°
Purple 280°	Green 100°

```
setColorHSV(h, 100, 100);  
DrawSwatch(10, 10, 20, 20);  
setColorHSV(h + 180, 100, 100);  
DrawSwatch(35, 10, 20, 20);
```

Side by side they give the **strongest contrast**. Mixed together they cancel to gray.

| Common pitfalls

Integer division

$i / 7$ is 0 for every i below 7. Write $i / 7.0$ to get a gray ramp.

Wrong scale

`glColor3f` wants 0 to 1. Our HSV code returns 0 to 255, so divide by 255.

Blending left on

Text and opaque shapes look wrong. Call `glDisable(GL_BLEND)` when you are done.

Background order

Draw backgrounds first, with blending off, then the blended circles.

Hue out of range

$h + 180$ can pass 360. Wrap it with `fmodf` and fix negative values.

Divide by zero

Gray and black pixels have $\text{diff} = 0$ or $\text{cmax} = 0$. Check before dividing.

Task 1: Gray ramp

- Draw 8 squares from black to white with a for loop.
- Use $\text{gray} = i / 7.0$ and `glColor3f(gray, gray, gray)`.
- Print each gray value under its square with `printSome`.

Hint

$R = G = B$ means you are walking along the main diagonal of the RGB cube. Use `sprintf` to turn the number into text.

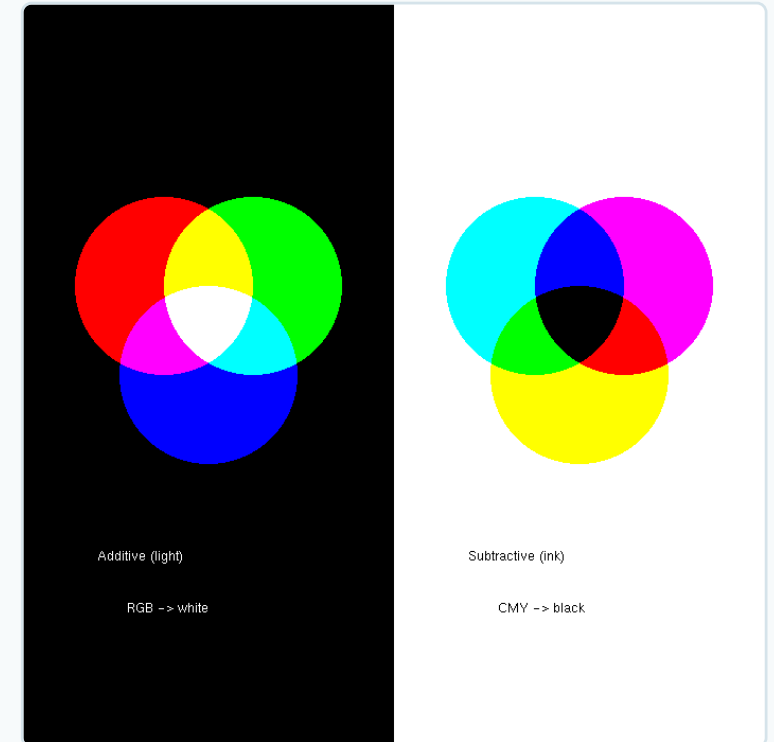


Task 2: Light versus ink

- Left half: black background with red, green and blue circles added together.
- Right half: white background with cyan, magenta and yellow circles multiplied.
- Label each half.

Hint

Draw both backgrounds with blending disabled, then enable it and switch `glBlendFunc` between the two halves.

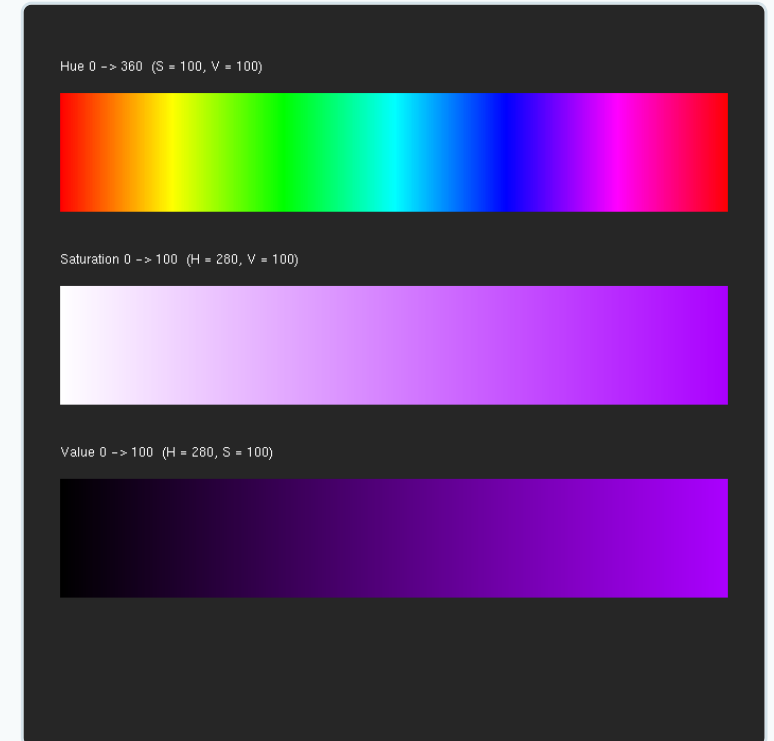


Task 3: HSV strips

- Write `rgbToHSV` and `hsvToRGB` and check them with the two worked examples.
- Draw three strips of 360 thin quads: hue 0 to 360, saturation 0 to 100 and value 0 to 100.
- Use `H = 280` for the last two strips and label each strip.

Hint

Make a helper `setColorHSV(h, s, v)` that calls `hsvToRGB` and then `glColor3f(r / 255, g / 255, b / 255)`.



Task 4: Sunset scene

- Sky: a quad that is dark blue at the top and orange at the horizon.
- Sun: a warm yellow circle, half hidden by the ground.
- Two mountains: triangles that are lighter at the peak.
- Ground: a quad from dark green to almost black. Write "Sunset" with `printSome`.

Hint

Draw back to front: sky, sun, mountains, ground. Per-vertex colors give you the gradients.



Bonus: rotate the hue

- Draw a circle in the current hue and a square in its complement.
- Press D to add 10° to the hue and A to subtract 10° .
- Print the current hue on screen.

Your display function is called again every time you call `glutPostRedisplay`.

```
float hue = 0;

void keyboard(unsigned char key, int x, int y)
{
    if (key == 'd') hue += 10;
    if (key == 'a') hue -= 10;
    glutPostRedisplay();
}

// in main(), before glutMainLoop():
glutKeyboardFunc(keyboard);
```

| Thanks

Questions?